

Hierarchical Hybrid RAG for Legal & Enterprise Documents

Project documentation — Phases 1-6 (ingestion through evaluation)

1. What this is

A retrieval-augmented generation (RAG) system purpose-built for complex legal and enterprise PDFs: nested clauses, cross-references, tables, and page boundaries. Unlike a basic chunk-and-embed pipeline, it preserves document hierarchy end to end and uses that structure during retrieval, reranking, and context assembly.

Pipeline stages

- 1 PDF parsing (layout-aware: font size, boldness, page, table geometry)
- 2 Structure detection (headings vs. body text; cross-reference extraction)
- 3 Hierarchical document tree (parent/child sections down to clause level)
- 4 Structure-aware chunking (clauses never split mid-way unless oversized)
- 5 Hybrid retrieval: dense (TF-IDF/SVD + FAISS) + lexical (BM25) + hierarchical tree walk
- 6 Reranking (feature-based cross-encoder-style scorer)
- 7 Relevance gate (blocks generation before it starts if nothing is relevant enough)
- 8 Context building (grouped, ordered, deduplicated, token-budgeted, cited)
- 9 Grounded generation (LLM call constrained to retrieved context only)
- 10 Groundedness verification with a bounded retry loop
- 11 Evaluation framework: 5-baseline ablation + RAG-level metrics

Folder layout

ingestion/ · hierarchy/ · schemas/ · config/ · vector_store/ · lexical_search/ · retrieval/ · reranking/ · context/ · generation/ · guardrails/ · evaluation/ · scripts/ (CLI entry points). Not yet built: api/ and ui/ (a FastAPI backend and web UI are Phase 7 — everything below runs from the command line today).

2. How to run it

2.1 Install dependencies (once)

```
pip install -r requirements.txt
```

2.2 Index a document (once per document)

```
python3 scripts/index_document.py path/to/your.pdf --name "My Document"  
→ saved to data/processed/<slug>/ (tree, chunks, dense index, lexical index)
```

2.3 Ask it questions interactively

```
python3 scripts/ask.py data/processed/<slug> --verbose
```

2.4 Evaluate it against a question set

```
python3 scripts/evaluate.py path/to/your.pdf your_questions.json --out evaluation/results/run.json
```

Question sets follow `evaluation/schema.py`. The key field is **expected_keyword_groups**: a list of keyword groups, where a retrieved chunk counts as relevant if it contains every keyword in *any one* group. Most questions need one group (one supporting passage); comparison questions can have two (either side of the comparison). “no_answer” questions use an empty list and `is_answerable`: false.

2.5 Set an LLM provider (optional — falls back to a deterministic mock otherwise)

```
export ANTHROPIC_API_KEY=sk-ant-... # or export GEMINI_API_KEY=...
```

Provider choice is automatic (`generation/providers.get_llm_provider`): Anthropic, then Gemini, then the mock. No code changes needed to switch.

3. Evaluation methodology & metrics

3.1 Five baselines (retrieval-only ablation)

#	Baseline	What it isolates
1	Fixed-size chunking + dense-only	The naive “chunk-and-embed” approach
2	Structure-aware chunking + dense-only	Value of structure-aware chunking alone
3	Hybrid (dense+BM25), no hierarchy	Value of adding lexical/keyword retrieval
4	Hybrid + hierarchical retrieval	Value of using the document tree
5	Hybrid + hierarchical + reranking	Value of the reranking stage — the full pipeline

Reported per baseline: Precision@5, Hit@5 (a “recall@5” in the single-supporting-passage sense — did at least one relevant chunk make the top 5), MRR, NDCG@5, mean latency, and mean tokens retrieved.

3.2 RAG-level metrics (full pipeline, by question category)

Metric	What it measures
Context precision	Of the chunks assembled into context, what fraction are actually relevant
Context recall	Did the assembled context include at least one supporting passage
Answer relevance	Vocabulary overlap between the answer and the question (on-topic check)
Faithfulness	Groundedness check passed — no fabricated citations or figures
Answer correctness	Fraction of expected keywords that appear in the generated answer
Answerability accuracy	Did the system correctly recognize answerable vs. unanswerable questions

3.3 A real, measured result (synthetic sample contract)

Baseline	Hit@5	Tokens retrieved
1. Fixed-size chunking, dense-only	0.87	1106
2. Structure-aware chunking, dense-only	1.00	237

Structure-aware chunking retrieves the right passage more reliably while using ~4.7x fewer tokens — the core architectural thesis, measured rather than asserted (rerun tests/test_evaluation.py to reproduce).

4. Real-world stress test

Tested against two real documents: a 186-page UK government “Short Form Contract” (converted from .odt via LibreOffice headless) and a 117-page Everus Construction Group 2025 10-K annual report. Findings, not smoothed over:

- Structure detection degrades on documents that don't use clean decimal numbering. The UK contract mixes Roman-numeral top sections with decimal sub-clauses; the 10-K uses “Item 1” / “PART I” conventions the heading detector has no pattern for.
- Two real bugs were caught and fixed during this test: bold financial-table column headers (“2025 2024”) and cover-page pull-quote figures (“\$3.75B”) were being misclassified as section headings before a guard was added.
- On the UK contract, hierarchical retrieval measurably hurt Hit@5 (0.78 vs. 0.89) — direct evidence that hierarchy only helps when the tree is a decent representation of the real structure.
- On the 10-K, hybrid (dense+BM25) retrieval clearly helped over dense-only (Hit@5 0.75 vs. 0.62).
- Cross-reference extraction found almost nothing in the UK contract: its hyperlinked cross-references show the target clause's title as the link text (“clause Data Protection and Security”), not a section number — a different convention than the pattern this system looks for.
- pdfplumber's default table detector over-fires on text-heavy PDFs without visible gridlines (75 “tables” detected in the 10-K, many of which are actually justified paragraph text).

5. Choosing an LLM provider: speed vs. accuracy vs. cost

Local model (Llama, Mistral, Qwen): no per-query cost, full data privacy, no external rate limits — but CPU-only inference is not fast (several seconds to tens of seconds even for a short answer). A GPU gets a quantized 7-8B model to ~1-3 seconds, but that means provisioning and maintaining a GPU inference server — real extra ops for a project whose actual contribution is the retrieval architecture, not the generation model.

Hosted fast-tier API model (Claude Haiku, Gemini Flash): consistently 1-3 seconds, zero infra, strong quality even at the cheap tier — because in RAG, retrieval quality does most of the work. **Recommendation: a hosted fast-tier model**, specifically because the local route only reaches “seconds” with a GPU this project doesn't otherwise need.

On free-tier Gemini rate limits: real, not hypothetical — evaluate.py fires one call per question back-to-back and will hit low free-tier quotas. Both AnthropicProvider and GeminiProvider (generation/providers.py) now retry on HTTP 429/503 with exponential backoff, but for a timed demo specifically, a small pay-as-you-go credit on Claude Haiku is more reliable than a free tier. Verify current published rate limits on the provider's own docs before a demo — they change often.

Neither provider has been live-tested in the development sandbox (no Anthropic key was available; generativelanguage.googleapis.com is outright blocked by that sandbox's network egress rules). Test both with real keys in your own environment before relying on either.

6. Known limitations (see README.md for the complete list)

- Dense retrieval uses TF-IDF+SVD, not a transformer embedding model (disk constraints in the dev sandbox); a real embedding model is a one-line swap in vector_store/embedder.py.
- Reranking is feature-based (phrase overlap, term recall/precision), not a trained cross-encoder, for the same reason.

- Groundedness checking is lexical-overlap-based, not an NLI/entailment model.
- The pre-generation relevance threshold is heuristic and has an observed failure mode: a short rephrasing of an unanswerable question can slip past it (quantified by the answerability_accuracy metric, not hidden).
- Evaluation ground truth (expected_keyword_groups) is written by hand per document.

7. Not yet built

Phase 7: input-sanitization hardening beyond the prompt-level defense already in place, a FastAPI backend. Phase 8: a web UI (Streamlit/React). Everything above already works end to end from the command line without either.